

INTERVIEW PREPARATION SERIES

Java Collections Framework

Interview Q&A — 40 Questions

This document covers **40 interview questions** spanning the Collections hierarchy, List (ArrayList, LinkedList, CopyOnWriteArrayList), Set (HashSet, LinkedHashSet, TreeSet, EnumSet), Map (HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap), generics, iterators, Comparable vs Comparator, immutable collections, streams, and concurrent collections. Designed for Java developers and full-stack engineers.

Collections Framework Overview

QUESTION 01

What is the Java Collections Framework? Describe the core interfaces and their hierarchy.

ANSWER

```
//          Iterable
//          |
//      Collection
//      /   |   \
//    List  Set  Queue
//          |   |
//      SortedSet Deque
//
//          Map (separate hierarchy)
//          |
//      SortedMap
```

- **Collection:** root interface for single-element collections. Methods: add, remove, contains, size, isEmpty, iterator, stream
- **List:** ordered, indexed, allows duplicates. Implementations: ArrayList, LinkedList, Vector
- **Set:** unordered (usually), no duplicates. Implementations: HashSet, LinkedHashSet, TreeSet
- **Queue/Deque:** FIFO/double-ended. Implementations: LinkedList, ArrayDeque, PriorityQueue

- **Map:** key-value pairs, unique keys. **Not** a Collection. Implementations: HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap

QUESTION 02

What is the difference between `Collection` and `Collections` in Java?

ANSWER

- **Collection:** an **interface** (`java.util.Collection`) — the root of the collection hierarchy
- **Collections:** a **utility class** (`java.util.Collections`) with static helper methods:

```
Collections.sort(list);
Collections.reverse(list);
Collections.unmodifiableList(list);
Collections.synchronizedMap(map);
Collections.singletonList("only");
Collections.emptyList();
Collections.frequency(list, element);
Collections.min(collection);
Collections.max(collection);
```

QUESTION 03

What are generics in collections? What is type erasure, and why does `List<String>` become `List` at runtime?

ANSWER

Generics provide compile-time type safety: `List<String>` guarantees only Strings. Without generics, you get `ClassCastException` at runtime.

```
// Without generics (raw type) – dangerous
List list = new ArrayList();
list.add("hello"); list.add(42); // no compile error!
String s = (String) list.get(1); // ClassCastException at RUNTIME

// With generics – safe
List<String> list = new ArrayList<>();
// list.add(42); // COMPILE ERROR
```

Type erasure: Java removes generic type information at compile time for backward compatibility. At runtime, `List<String>` and `List<Integer>` are both just `List`. This means you cannot do `instanceof List<String>` or create `new T[]()`.

List Implementations

QUESTION 04

What is `ArrayList`? How does it work internally, and what are its time complexities?

ANSWER

`ArrayList` is a **resizable array**. Internally, it uses an `Object[]` that doubles when full (actually grows by 50% in modern JDK).

- **`get(i)` / `set(i, e)`:** $O(1)$ — direct array index
- **`add(e)` (at end):** $O(1)$ amortized — occasional resize
- **`add(i, e)` (at index):** $O(n)$ — shifts elements right
- **`remove(i)`:** $O(n)$ — shifts elements left
- **`contains(e)`:** $O(n)$ — linear search

Initial capacity: 10. Use `new ArrayList<>(expectedSize)` to avoid resizing.

`ArrayList` is the **default choice** for most List use cases.

QUESTION 05

What is `LinkedList`? When should you use it instead of `ArrayList`?

ANSWER

`LinkedList` is a **doubly linked list**. Each element is a Node with prev/next pointers.

- **`addFirst` / `addLast`:** $O(1)$
- **`get(i)`:** $O(n)$ — must traverse from head or tail
- **`add(i, e)`:** $O(n)$ to find position, $O(1)$ to insert
- **`remove via iterator`:** $O(1)$ — just relink pointers

Use LinkedList when: you frequently add/remove at the **beginning** of the list, or you need Deque operations (addFirst, removeLast). In practice, **ArrayList outperforms LinkedList in almost all scenarios** due to cache locality.

Java's own recommendation: use `ArrayDeque` instead of LinkedList for Deque operations.

QUESTION 06

What is `Vector`? How does it differ from `ArrayList`, and why is it considered legacy?

ANSWER

`Vector` is a **synchronized version of `ArrayList`**. Every method (add, get, remove) is `synchronized`, making it thread-safe.

Why it is legacy:

- Synchronizing every method is too coarse-grained. Modern concurrent access uses `CopyOnWriteArrayList` or external synchronization
- `Vector` doubles capacity (2x) on resize; `ArrayList` grows by 50% (more memory-efficient)
- Use `Collections.synchronizedList(new ArrayList<>())` or `CopyOnWriteArrayList` instead

QUESTION 07

What is `CopyOnWriteArrayList`? How does it achieve thread safety, and when should you use it?

ANSWER

`CopyOnWriteArrayList` creates a **new copy of the internal array on every write** (add, set, remove). Reads are lock-free.

- **Read:** $O(1)$, no synchronization needed
- **Write:** $O(n)$ — copies entire array

Use when: reads vastly outnumber writes (event listener lists, configuration caches). Do **not** use for frequently modified lists.

Set Implementations

QUESTION 08

What is `HashSet`? How does it guarantee uniqueness, and what are the requirements for objects stored in it?

ANSWER

`HashSet` is backed by a `HashMap` internally. Elements are stored as keys; values are a constant dummy object.

Uniqueness guarantee: before adding, `HashSet` calls `hashCode()` to find the bucket, then `equals()` to check if the element already exists.

Requirements for stored objects:

- Must implement `hashCode()` and `equals()` consistently
- If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **must** be true
- Objects should be **immutable** after insertion (changing a field used in `hashCode` breaks the set)

```
Set<String> unique = new HashSet<>();
unique.add("hello");
unique.add("hello"); // ignored – equals returns true
System.out.println(unique.size()); // 1
```

QUESTION 09

What is the difference between `HashSet`, `LinkedHashSet`, and `TreeSet`?

ANSWER

- **HashSet:** $O(1)$ add/remove/contains. **No ordering guarantee.** Backed by `HashMap`.
- **LinkedHashSet:** $O(1)$ operations like `HashSet`, but maintains **insertion order**. Uses a linked list through the hash table entries.
- **TreeSet:** $O(\log n)$ operations. Maintains elements in **sorted order** (natural ordering or `Comparator`). Backed by a Red-Black tree (`TreeMap`).

```
Set<Integer> hash = new HashSet<>(List.of(5,3,1,4,2));
// Iteration order: unpredictable (e.g., 1,2,3,4,5 or 5,4,3,2,1)

Set<Integer> linked = new LinkedHashSet<>(List.of(5,3,1,4,2));
```

```
// Iteration order: 5,3,1,4,2 (insertion order preserved)

Set<Integer> tree = new TreeSet<>(List.of(5,3,1,4,2));
// Iteration order: 1,2,3,4,5 (sorted)
```

QUESTION 10

What is `EnumSet`? Why is it the most efficient Set implementation for enum types?

ANSWER

`EnumSet` is a specialized Set for enum constants. Internally uses a **bit vector** — each enum constant is represented by a single bit.

```
enum Permission { READ, WRITE, EXECUTE, DELETE }

EnumSet<Permission> admin = EnumSet.allOf(Permission.class);
EnumSet<Permission> reader = EnumSet.of(Permission.READ);
EnumSet<Permission> editor = EnumSet.of(Permission.READ, Permission.WRITE);
```

Performance: all operations (add, contains, remove) are **O(1) bitwise operations**. Extremely memory-efficient (one long for up to 64 enum constants). **Always use `EnumSet` instead of `HashSet` for enum types.**

Map Implementations

QUESTION 11

What is `HashMap`? How does it work internally (hashing, buckets, collisions, resizing)?

ANSWER

`HashMap` stores key-value pairs in an **array of buckets**. Each bucket holds a linked list (or tree for ≥ 8 entries) of entries.

How it works:

- Put: compute `key.hashCode()` → bucket index = hash % capacity → store entry
- Get: compute hash → find bucket → walk the chain, compare with `equals()`

- Collision: two keys hash to the same bucket. Resolved by chaining (linked list, converted to **Red-Black tree at 8 entries**)
- Resize: when load factor (0.75 default) is exceeded, the array doubles and all entries are rehashed

Complexities: $O(1)$ average for get/put/remove. $O(n)$ worst case (all in one bucket). $O(\log n)$ worst case with treeification (Java 8+).

QUESTION 12

What is the difference between `HashMap`, `LinkedHashMap`, `TreeMap`, and `Hashtable`?

ANSWER

- **HashMap:** $O(1)$ operations. No ordering. Allows one null key and multiple null values. Not thread-safe.
- **LinkedHashMap:** $O(1)$ operations + **insertion order** (or access order for LRU caches). Slightly more memory than HashMap.
- **TreeMap:** $O(\log n)$ operations. **Sorted by key** (natural order or Comparator). Backed by Red-Black tree.
- **Hashtable: Legacy.** Synchronized (every method). No null keys or values. Use `ConcurrentHashMap` instead.

```
// LRU cache with LinkedHashMap (access-order mode)
Map<String, Data> lru = new LinkedHashMap<>(16, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry eldest) {
        return size() > MAX_CACHE_SIZE;
    }
};
```

QUESTION 13

What is `ConcurrentHashMap`? How does it achieve thread safety without locking the entire map?

ANSWER

`ConcurrentHashMap` uses **lock striping** (Java 7: segments) and **CAS (Compare-And-Swap)** operations (Java 8+) to allow concurrent reads and writes without a global lock.

- **Reads:** fully lock-free, always consistent

- **Writes:** lock only the specific bucket being modified
- **Iteration:** weakly consistent — reflects some but not all concurrent modifications
- **No null keys or values:** null is used internally as a sentinel

```
ConcurrentHashMap<String, Integer> counters = new ConcurrentHashMap<>();

// Atomic operations
counters.putIfAbsent("views", 0);
counters.compute("views", (key, val) -> val + 1); // atomic increment
counters.merge("views", 1, Integer::sum);         // atomic merge
```

QUESTION 14

What is the `hashCode()` and `equals()` contract? What happens if you violate it with `HashMap`/`HashSet`?

ANSWER

The contract:

- If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **must** be true
- If `a.hashCode() == b.hashCode()`, `a.equals(b)` may or may not be true (collision is OK)

Violations:

- **Override equals but not hashCode:** equal objects hash to different buckets. **HashSet allows “duplicate” objects.** `HashMap.get()` returns null for a key that exists.
- **Override hashCode but not equals:** objects with same hash are compared with `Object.equals()` (identity). Logically equal objects are treated as different.

```
// CORRECT implementation
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User)) return false;
    User u = (User) o;
    return id == u.id && Objects.equals(name, u.name);
}
@Override public int hashCode() {
    return Objects.hash(id, name); // same fields as equals
}
```


Iteration, Sorting & Utilities

QUESTION 15

What is `Iterator` vs `ListIterator`? What is `ConcurrentModificationException` and how do you avoid it?

ANSWER

- **Iterator:** forward-only traversal. Methods: `hasNext()`, `next()`, `remove()`. Works on any Collection.
- **ListIterator:** bidirectional (`hasPrevious/previous`). Can add, set, and get index. Only for Lists.

ConcurrentModificationException: thrown when a collection is modified during iteration (not via the iterator's own methods).

```
// WRONG: modifying list during for-each
for (String s : list) {
    if (s.equals("remove")) list.remove(s); //
    ConcurrentModificationException!
}

// CORRECT: use Iterator.remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("remove")) it.remove(); // safe
}

// CORRECT: Java 8+ removeIf
list.removeIf(s -> s.equals("remove"));
```

QUESTION 16

What is the difference between `Comparable` and `Comparator`? How do you sort collections with each?

ANSWER

```
// Comparable: natural ordering defined in the class itself
public class Employee implements Comparable<Employee> {
    public int compareTo(Employee other) {
        return this.name.compareTo(other.name); // sort by name
    }
}

Collections.sort(employees); // uses compareTo
```

```
// Comparator: external ordering, multiple possible
Comparator<Employee> bySalary = Comparator.comparing(Employee::getSalary);
Comparator<Employee> byNameDesc =
    Comparator.comparing(Employee::getName).reversed();
Comparator<Employee> byDeptThenSalary = Comparator
    .comparing(Employee::getDepartment)
    .thenComparing(Employee::getSalary);

employees.sort(bySalary);
employees.sort(byDeptThenSalary);
```

Rule: implement Comparable for the **one natural ordering** (alphabetical, chronological). Use Comparator for **alternative orderings**.

QUESTION 17

How do you convert between collection types? List to Set, Set to List, Array to List, etc.

ANSWER

```
// List to Set (removes duplicates)
Set<String> set = new HashSet<>(list);

// Set to List
List<String> list = new ArrayList<>(set);

// Array to List
List<String> list = Arrays.asList(array);           // fixed-size!
List<String> list = new ArrayList<>(Arrays.asList(array)); // mutable
List<String> list = List.of(array);                 // immutable (Java 9+)

// List to Array
String[] array = list.toArray(new String[0]);

// Stream conversions
List<String> filtered = set.stream().filter(s -> s.length() >
    3).collect(Collectors.toList());
```

QUESTION 18

What are immutable collections in Java? How do `List.of()`, `Set.of()`, and `Map.of()` work?

ANSWER

```
// Java 9+: immutable factory methods
List<String> list = List.of("a", "b", "c");
Set<Integer> set = Set.of(1, 2, 3);
Map<String, Integer> map = Map.of("one", 1, "two", 2);

// Any modification throws UnsupportedOperationException
list.add("d");      // UnsupportedOperationException!
set.remove(1);      // UnsupportedOperationException!

// Null is NOT allowed
List.of(null);      // NullPointerException!

// Older approach: Collections.unmodifiableList
List<String> unmod = Collections.unmodifiableList(new
ArrayList<>(original));
```

Use immutable collections for: constants, method return values (prevents callers from modifying internal state), and thread safety.

QUESTION 19

What is `PriorityQueue` in Java? How does it differ from a regular Queue?

ANSWER

`PriorityQueue` is a **min-heap**. The element with the lowest value (or highest priority per `Comparator`) is always at the head.

```
// Min-heap (default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.addAll(List.of(5, 1, 3, 2, 4));
minHeap.poll(); // returns 1 (smallest)

// Max-heap
PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Comparator.reverseOrder());

// Custom priority
PriorityQueue<Task> taskQueue = new PriorityQueue<>(
    Comparator.comparing(Task::getPriority));

// Complexities: offer/poll = O(log n), peek = O(1), contains = O(n)
```

QUESTION 20

What is the `Map.Entry` interface? How do you iterate over a `Map` efficiently?

ANSWER

```
Map<String, Integer> map = Map.of("a", 1, "b", 2, "c", 3);

// Method 1: entrySet (MOST EFFICIENT for accessing key+value)
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}

// Method 2: keySet (extra lookup per key)
for (String key : map.keySet()) {
    System.out.println(key + " = " + map.get(key)); // extra O(1) lookup
}

// Method 3: forEach (Java 8+)
map.forEach((key, value) -> System.out.println(key + " = " + value));

// Method 4: Stream
map.entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .forEach(e -> System.out.println(e.getKey()));
```

QUESTION 21

What is `ArrayDeque` and why is it preferred over `LinkedList` for Stack and Queue operations in Java?

ANSWER

`ArrayDeque` is a **resizable circular array** implementing `Deque`. It is the recommended replacement for both `Stack` and `LinkedList` for LIFO/FIFO operations.

- **vs Stack:** `Stack` is synchronized (unnecessary overhead) and extends `Vector` (poor design)
- **vs LinkedList:** `ArrayDeque` has better cache locality, no per-element node overhead, and faster operations

```
// As a Stack
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.pop(); // 2
```

```
// As a Queue
Deque<Integer> queue = new ArrayDeque<>();
queue.offer(1); queue.offer(2); queue.poll(); // 1
```

QUESTION 22

What is the `Collections.unmodifiableList()` gotcha? Why might it not truly protect your data?

ANSWER

```
List<String> original = new ArrayList<>(List.of("a", "b"));
List<String> unmod = Collections.unmodifiableList(original);

unmod.add("c"); // UnsupportedOperationException ✓

// BUT: the original can still be modified!
original.add("c"); // works!
System.out.println(unmod); // [a, b, c] – unmod reflects the change!
```

Fix: create a **defensive copy**: `Collections.unmodifiableList(new ArrayList<>(original))`. Or use `List.copyOf(original)` (Java 10+) which creates an independent copy.

QUESTION 23

How do you use `computeIfAbsent`, `computeIfPresent`, and `merge` on a `Map`?

ANSWER

```
Map<String, List<String>> groups = new HashMap<>();

// computeIfAbsent: initialize if key missing
groups.computeIfAbsent("fruits", k -> new ArrayList<>()).add("apple");

// computeIfPresent: modify only if key exists
map.computeIfPresent("key", (k, v) -> v + 1);

// merge: combine old and new values
Map<String, Integer> counts = new HashMap<>();
for (String word : words) {
    counts.merge(word, 1, Integer::sum); // count words
}
```

These methods are **atomic for ConcurrentHashMap**, making them the correct way to do check-then-act operations on concurrent maps.

QUESTION 24

What is the difference between `fail-fast` and `fail-safe` iterators?

ANSWER

- **Fail-fast:** iterators from `ArrayList`, `HashMap`, `HashSet`. Throw `ConcurrentModificationException` if the collection is modified during iteration. Detect via a modification counter.
- **Fail-safe:** iterators from `CopyOnWriteArrayList`, `ConcurrentHashMap`. Iterate over a snapshot or use internal synchronization. Never throw `ConcurrentModificationException`.

```
// Fail-fast
List<String> list = new ArrayList<>(List.of("a","b","c"));
for (String s : list) { list.add("d"); } //
ConcurrentModificationException

// Fail-safe
List<String> cowList = new CopyOnWriteArrayList<>(List.of("a","b","c"));
for (String s : cowList) { cowList.add("d"); } // OK (iterates snapshot)
```

QUESTION 25

What is `NavigableMap` and `NavigableSet`? What additional operations do `TreeMap` and `TreeSet` provide?

ANSWER

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(10, "A"); map.put(20, "B"); map.put(30, "C"); map.put(40, "D");

map.firstKey();      // 10
map.lastKey();       // 40
map.floorKey(25);    // 20 (greatest key <= 25)
map.ceilingKey(25);  // 30 (smallest key >= 25)
map.headMap(30);     // {10=A, 20=B} (keys < 30)
map.tailMap(20);     // {20=B, 30=C, 40=D} (keys >= 20)
map.subMap(15, 35);  // {20=B, 30=C} (range query)
```

These range operations make TreeMap/TreeSet ideal for: scheduling (find next event), time-series data, and range queries.

QUESTION 26

What is IdentityHashMap? How does it differ from HashMap?

ANSWER

IdentityHashMap uses **reference equality (==)** instead of `equals()` for key comparison. Two keys are considered equal only if they are the **same object**.

```
Map<String, Integer> identity = new IdentityHashMap<>();
String a = new String("hello");
String b = new String("hello");
identity.put(a, 1); identity.put(b, 2);
identity.size(); // 2! (a and b are different objects)

Map<String, Integer> regular = new HashMap<>();
regular.put(a, 1); regular.put(b, 2);
regular.size(); // 1 (a.equals(b) is true)
```

Use cases: serialization frameworks, graph traversal (tracking visited objects by identity), and proxying.

QUESTION 27

What is WeakHashMap? How does it interact with garbage collection?

ANSWER

WeakHashMap stores keys as **weak references**. When a key has no strong references elsewhere, it is garbage collected, and the entry is automatically removed from the map.

```
WeakHashMap<Object, String> cache = new WeakHashMap<>();
Object key = new Object();
cache.put(key, "data");
System.out.println(cache.size()); // 1
key = null; // no more strong references to the key
System.gc(); // key is eligible for GC
System.out.println(cache.size()); // 0 (entry removed!)
```

Use cases: caches where entries should be evicted when the key is no longer in use. ClassLoader caches, metadata caches.

QUESTION 28

How do Java Streams interact with collections? What are the key terminal and intermediate operations?

ANSWER

```
List<Employee> employees = getEmployees();

// Intermediate (lazy): filter, map, flatMap, sorted, distinct, peek
// Terminal (triggers execution): collect, forEach, reduce, count, min, max

// Example: get names of employees earning > 80k, sorted
List<String> highEarnings = employees.stream()
    .filter(e -> e.getSalary() > 80000)
    .sorted(Comparator.comparing(Employee::getName))
    .map(Employee::getName)
    .collect(Collectors.toList());

// Grouping by department
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Sum salaries
double totalSalary = employees.stream()
    .mapToDouble(Employee::getSalary).sum();
```

QUESTION 29

What are `Collections.synchronizedList/Map/Set`? How do they compare to concurrent collections?

ANSWER

- **synchronizedXxx:** wraps a collection with a mutex. All methods are synchronized on the same lock. Simple but coarse — only one thread can access at a time.
- **Concurrent collections:** purpose-built for concurrency. `ConcurrentHashMap` uses lock striping. `CopyOnWriteArrayList` uses copy-on-write. **Much higher throughput** under concurrent access.

Caveat: synchronized wrappers require external synchronization for compound operations (check-then-act):

```
List<String> syncList = Collections.synchronizedList(new ArrayList<>());

// WRONG: not atomic
if (!syncList.contains("item")) syncList.add("item");

// RIGHT: synchronize the compound operation
synchronized (syncList) {
    if (!syncList.contains("item")) syncList.add("item");
}
```

QUESTION 30

What is the `Deque` interface? How does `ArrayDeque` implement it, and what makes it special?

ANSWER

`Deque` (double-ended queue) supports $O(1)$ operations at both ends.

```
Deque<Integer> deque = new ArrayDeque<>();
deque.offerFirst(1); // [1]
deque.offerLast(2); // [1, 2]
deque.offerFirst(0); // [0, 1, 2]
deque.pollFirst(); // 0 (returns and removes)
deque.pollLast(); // 2
deque.peekFirst(); // 1 (read without remove)
```

`ArrayDeque` is the **recommended general-purpose `Deque`**: faster than `LinkedList` (cache-friendly), no capacity restrictions (resizes), and no null elements (uses null as sentinel).

QUESTION 31

How do you create a thread-safe `Set` in Java?

ANSWER

```
// Option 1: ConcurrentHashMap-backed set (best for high concurrency)
Set<String> concurrentSet = ConcurrentHashMap.newKeySet();

// Option 2: CopyOnWriteArraySet (best for read-heavy)
Set<String> cowSet = new CopyOnWriteArraySet<>();
```

```
// Option 3: Synchronized wrapper  
Set<String> syncSet = Collections.synchronizedSet(new HashSet<>());
```

QUESTION 32

What is `EnumMap`? Why is it faster than `HashMap` for enum keys?

ANSWER

`EnumMap` uses the enum's **ordinal as the array index**. No hashing, no collision resolution, no boxing.

```
EnumMap<DayOfWeek, String> schedule = new EnumMap<>(DayOfWeek.class);  
schedule.put(DayOfWeek.MONDAY, "Meeting");  
schedule.put(DayOfWeek.FRIDAY, "Deploy");
```

Performance: $O(1)$ with a simple array lookup. **Always use `EnumMap` over `HashMap` when keys are enums.**

QUESTION 33

What is `Collections.singletonList()` vs `List.of()`? When do you use single-element collections?

ANSWER

```
List<String> singleton = Collections.singletonList("only"); // immutable,  
1 element  
List<String> listOf = List.of("only"); // immutable,  
Java 9+  
  
// Both are immutable. List.of() is preferred (Java 9+)  
// singletonList allows null; List.of does not
```

Use for: method parameters that expect a `List` but you have one element, default return values, and test fixtures.

QUESTION 34

How does `HashMap` handle null keys and values? What about `TreeMap` and `ConcurrentHashMap`?

ANSWER

- **HashMap:** allows **one null key** (hashes to bucket 0) and **multiple null values**
- **TreeMap:** **no null keys** (`compareTo/Comparator` cannot handle null). Null values are allowed
- **ConcurrentHashMap:** **no null keys or values** (null is used as a sentinel internally)
- **Hashtable:** no null keys or values

QUESTION 35

What is the `Splitter` and how does it enable parallel stream processing?

ANSWER

`Splitter` (`Splittable Iterator`) allows a collection to be split into chunks for **parallel processing**.

```
// Parallel stream uses Splitter internally
long count = list.parallelStream().filter(s -> s.length() > 3).count();

// Custom Splitter for a data source
Splitter<String> splitter = list.splitter();
Splitter<String> secondHalf = splitter.trySplit(); // split in two
```

Each collection provides its own `Splitter` that knows how to efficiently split its data. `ArrayList` splits by index range; `TreeSet` splits by subtrees.

QUESTION 36

What are the `Queue` methods `add/offer`, `remove/poll`, and `element/peek`? How do they differ?

ANSWER

- **`add()` vs `offer()`:** both enqueue. `add()` throws `IllegalStateException` if full. `offer()` returns false.
- **`remove()` vs `poll()`:** both dequeue. `remove()` throws `NoSuchElementException` if empty. `poll()` returns null.

- **element() vs peek():** both read head. `element()` throws if empty. `peek()` returns null.

Convention: use **offer/poll/peek** (return special value) for bounded queues. Use **add/remove/element** when failure is a bug.

QUESTION 37

How do you create a fixed-size cache using `LinkedHashMap` with access-order eviction (LRU)?

ANSWER

```
public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxSize;

    public LRUCache(int maxSize) {
        super(16, 0.75f, true); // true = access-order (not insertion)
        this.maxSize = maxSize;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > maxSize; // auto-remove when exceeded
    }
}

LRUCache<String, Data> cache = new LRUCache<>(100);
cache.put("key", data);
cache.get("key"); // moves to most-recently-used position
```

QUESTION 38

What are `Map.getOrDefault()`, `putIfAbsent()`, and `replace()`? How do they simplify Map operations?

ANSWER

```
Map<String, Integer> map = new HashMap<>();

// getOrDefault: avoid null checks
int count = map.getOrDefault("key", 0); // returns 0 if absent

// putIfAbsent: set only if not present
map.putIfAbsent("key", 1); // sets to 1 only if key is absent
```

```
// replace: update only if present
map.replace("key", 42);    // updates only if key exists
map.replace("key", 1, 42); // updates only if current value is 1
```

These methods eliminate common check-then-act patterns and are **atomic for ConcurrentHashMap**.

QUESTION 39

What is the performance comparison of the main collection implementations?

ANSWER

```
// List: get/set  add(end)  add(mid)  contains
// ArrayList:   0(1)      0(1)*    0(n)    0(n)
// LinkedList:   0(n)      0(1)    0(1)**  0(n)
//
// Set:
// HashSet:      0(1)      0(1)    0(1)    no
// LinkedHS:     0(1)      0(1)    0(1)    insertion
// TreeSet:      0(lgn)    0(lgn)  0(lgn)  sorted
//
// Map:
// HashMap:      0(1)      0(1)    0(1)    no
// LinkedHM:     0(1)      0(1)    0(1)    insertion/access
// TreeMap:      0(lgn)    0(lgn)  0(lgn)  sorted
// ConCHM:       0(1)      0(1)    0(1)    no (thread-safe)
//
// * amortized  ** with iterator position
```

QUESTION 40

Summarize the decision guide: which collection implementation should you choose for each use case?

ANSWER

- **General-purpose list:** ArrayList
- **Unique elements:** HashSet
- **Unique + insertion order:** LinkedHashSet
- **Unique + sorted:** TreeSet

-
- **Key-value lookup:** HashMap
 - **Key-value + insertion order:** LinkedHashMap
 - **Key-value + sorted keys:** TreeMap
 - **Thread-safe map:** ConcurrentHashMap
 - **Thread-safe list (read-heavy):** CopyOnWriteArrayList
 - **FIFO queue:** ArrayDeque
 - **Priority queue:** PriorityQueue
 - **Stack:** ArrayDeque (not Stack class)
 - **Enum keys:** EnumMap / EnumSet
 - **LRU cache:** LinkedHashMap with access-order
-